So in a distributed system like blockchain, verification is the primary defense against malicious users. Every node must independently verify every transaction to ensure the system remains “trustless” as there is no overarching authority.

So we need to implement Cryptographic Verification

Cryptographic Verification

Before checking if a user has the money, the system will check if the transaction is authentic and unaltered.

1. Digital Signatures (ECDSA)
    a. Math
        i. Most blockchains use Elliptic Curve Cryptography, where a user signs a transaction hash H(m) with their private key d_A to produce a signature pair (r, s)
    b. Check
        i. To verify, a node uses the transaction data m, signature (r, s), and senders public key Q_A. Node will then mathematically reconstruct a point on the elliptic curve. If the point aligns with random value r generated during signing, then the signature is valid
    c. Proof
        i. This will prove that the following:
            1. Ownership – only holder of private key could have generated (r, s) for specific message m
            2. Non-repudiation – sender cannot later deny they sent it
            3. Integrity – if one bit of transaction data m is changed, then the hash H(m) changes, causing mathematical verification to fail.
2. Hashing (SHA256)
    a. Every transaction is identified by its Transaction ID (TxID), which is the crypto hash of its data
    b. Verification – nodes re-hash the received transaction data to ensure calculated hash matches the provided TxID. This ensures data wasn’t corrupted during ‘network transmission’.

Logical & Semantic Verification (The "What" and "validity")

Once signature is verified, the node must check if the transaction makes sense within the current state of the ledger. Since our project more so aligns Ethereum, we’ll use the Account Model. The project uses accounts like a bank, so verification focuses on global state.

1. Account Model
    a. None Check (for replay attacks) – every transaction from an account must have a sequential number (Nonce)
        i. We’ll check: Tx.None == Account[Sender].Nonce.
        ii. If Tx.Nonce is too low, it’s a replay (old TX). If too high, out of order.
    b. Balance Check – does Account[Sender].Balance >= Tx.Value + Tx.Fee.
        i. NOTE – Don’t think we’ll integrate a Tx.Fee for simplicity but I’ll talk to Caleb about this.

Integrating Verification into Parallel PBFT

In PBFT implementation, verification happens at specific stages to stop faulty data from propagating.

PBFT Phase (sequential order)	Verification Action	System Behavior
Client Request	Primary node performs 'gatekeeper' check	Primary Leader validates transaction immediately upon receipt. If invalid, drops TX and doesn't propose in a block
Pre-Prepare	Backup nodes validate Proposal	When backups receive a PRE-PREPARE message containing a block from the Primary, they run the full verification suite (Crypto + Logical) on every transaction in that block.
Prepare	Consensus on Validity	If backup node finds TX is faulty, it refuses to broadcast a PREPARE message (or broadcasts a VIEW-CHANGE vote accusing leader of being malicious NOTE – Block only moves forward if 2f + 1 nodes agree it's valid
Commit	State Update	No verification here. Nodes check if they have received enough PREPARE votes from others to finalize block

How to Parallelize This

1. Parallel Signature Verification
    a. Bottleneck – Verifying ECDSA signatures is expensive
    b. Idea – In Pre-Prepare phase, a node receives a block with 1000+ transactions (depending on simulation setup, will very different sizes). Instead of verifying them sequentially, they'll be given to a thread pool (OpenMP) or GPU
2. Sharding Verification
    a. Node A only verifies transcations for Shard 1 and assumes Shard 2 transactions are verified by Node B. This will split verification workload by N shards

Public/Private Key Generation

Since we'll be simulating a massive amount of transactions, we need a efficient way to compute key generations. We'll utilize secp256k1, which is industry standard and used by Bitcoin and Ethereum.

1. Concept (Elliptic Curve Cryptography
    a. Private key (d) – cryptographic secure random 256-bit integer
    b. Public key (Q) – point on elliptic curve calculated by multiplying the private key with a generator point G.
        i. Q = d * G
    c. Curve: use secp256k1 since it's standard
2. Implementation in C++
    a. We'll use OpenSSL, since this is what actual blockchain clients use for their underlying crypto operations
    b. Prereq – Link against OpenSSL
    c. Integration
        i. For PBFT, we'll verify transactions in parallel
            1. Generation – use C++ script to pre-process and generate a file 'keys.json' with X number of pairs, based on configuration for simulation
            2. Simulation – load the file into nodes
            3. Parallel Check – when a node receives a block of Y transactions, it'll distribute the verification of signatures (using public keys) across available processes.
3. Resources for secp256k1
    a. https://en.bitcoin.it/wiki/Secp256k1
    b. https://github.com/bitcoin-core/secp256k1
    c. https://www.nervos.org/knowledge-base/secp256k1_a_key%20algorithm_(explainCKBot)

Using Public/Private Keys to Verify Signatures for Transactions

To verify signatures, we'll implement a lock and key mechanism using ECDSA. Process is as follows:

1. Concept – Fingerprint – Never sign or verify entire TX obj directly, instead verify a hash of TX.data.
    a. Sender (private key) – creates a hash of TX.data and encrypts that has with their private key to produce the signature
    b. Verifier (public key) – takes same transaction data, hashes it, and uses sender's public key and signature to run mathematical check
    c. End Result – if math aligns, proves two things
        i. Authenticity
        ii. Integrity
2. Data Prep (Serialization)
    a. Before verifying, must ensure verifier is looking at same data sender signed
        i. Rule – concatenate fields (sender, receiver, amount, nonce) into byte array
        ii. Exclude signature – this can't be signed, it's hashed

Workflow for Handling All of this

So since our project is targeting improving Time and Throughput, we're going to perform all of the crypto generation before the actual simulation takes place.

Steps:

1. Setup Phase (Offline – not time measured)
    a. Generate wallet pool – create a file containing X pre-generated public/private key pairs
    b. Generate a transaction pool – create a large dataset of valid, signed transaction objects
2. Simulation Phase (Online – time measured)
    a. Load data (initialization)
        i. Each simulated node loads wallets.json (for public keys) and a batch of transactions from workload.dat into memory
    b. Work (Parallel Verification)
        i. When a block proposal event occurs, the node iterates through the transactions
        ii. Inject verifyTransaction() function
3. Handling Faulty TXs in Simulation
    a. To prove that our system works, we'll intentially generate X% bad data during Offline Setup Phase
        i. Poison TX – in our TX generator script, we'll occasionally flip a bit in data after signing it, or sign it with the wrong private key
        ii. Tagging – we'll mark there in our logs
        iii. Verification – during simulation, we'll make sure to log out the reason for a TX being faulty
        iv. Success Metric – if no faulty TXs make it to the blockchain